

NAME :S.GAYATHERI

COURSE CODE : CSA1013

REG NO: 192521306

COURSE NAME : SOFTWARE

ENGINEERING

CO4-ASSESSMENT TOOL - 02

Title: AI-Powered Personalized Nutrition and Health Recommendation Platform

1. Project Overview

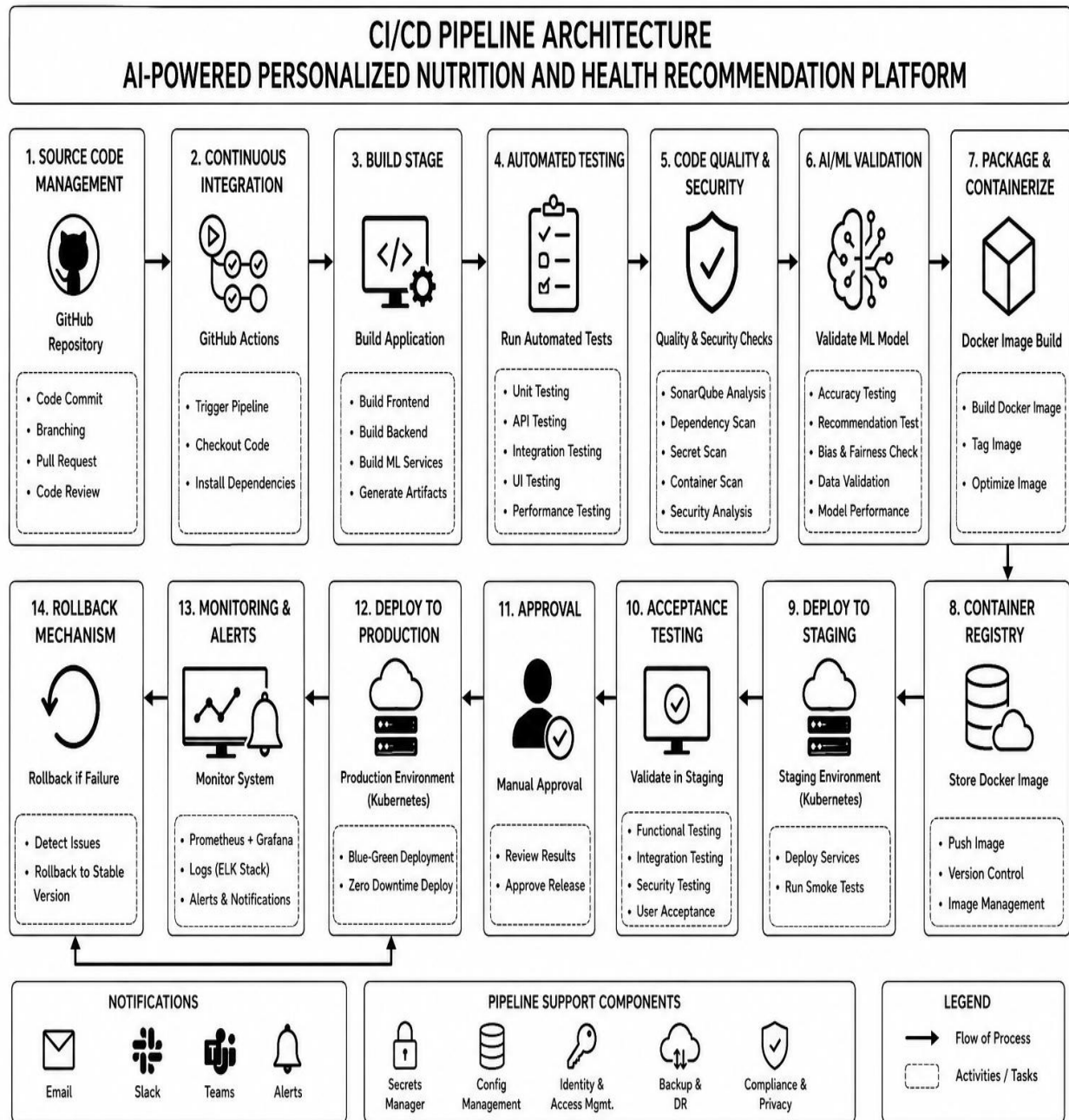
The proposed platform is an AI-powered health and nutrition system that collects personal health information, dietary preferences, physical activity, medical history, nutritional intake, and wearable-device data.

The system uses AI to generate personalized nutrition recommendations and meal plans, monitor dietary progress, and provide continuous health insights. Because the platform handles sensitive health information and uses AI-based recommendations, the CI/CD pipeline must emphasize automated testing, security, privacy, model validation, quality checks, controlled deployment, monitoring, and rollback.

2. CI/CD Requirements Analysis

Project Requirement	CI/CD Requirement
Personal health information	Security and privacy testing
AI nutrition recommendations	ML model validation
Personalized meal plans	Recommendation testing
Wearable integration	API and integration testing
Mobile/web application	UI and functional testing
Health-data processing	Data validation testing
Continuous monitoring	Production monitoring
Frequent software updates	Automated CI/CD
Sensitive user data	Secret management and encryption
Reliable recommendations	Quality gates and model approval
Production reliability	Staging environment and rollback

3. CI/CD Pipeline Architecture



4. Tools and Technology Selection

CI/CD Stage	Recommended Tool	Purpose
Source Control	GitHub	Version control
CI/CD	GitHub Actions	Pipeline automation
Backend	Python / FastAPI	Health and nutrition APIs

Frontend	React / Flutter	Web/mobile interface
Testing	PyTest	Automated Python tests
API Testing	Postman/Newman	API validation
UI Testing	Selenium	Application UI testing
Code Quality	SonarQube	Code quality analysis
Security	Trivy + OWASP	Vulnerability scanning
Secret Detection	GitHub Secret Scanning	Detect exposed secrets
ML Validation	PyTest + MLflow	Model testing and tracking
Containerization	Docker	Application packaging
Registry	AWS ECR / Docker Registry	Store images
Deployment	Kubernetes	Container orchestration
Cloud	AWS / Azure	Infrastructure
Monitoring	Prometheus	Metrics
Dashboard	Grafana	Monitoring visualization
Logging	ELK Stack	Centralized logs
Notifications	Email / Slack	Pipeline alerts

5. Pipeline Stages

Stage 1 – Source Code Management

All source code is maintained in GitHub.

The repository may contain:

nutrition-platform/

|

|— frontend/

|— backend/

|— ml-model/

|— wearable-service/

|— database/

|— tests/
|— Dockerfile
└— .github/
 └— workflows/

Developers use feature branches.

Feature Branch

↓

Pull Request

↓

Code Review

↓

Main Branch

↓

CI/CD Pipeline

6. Stage 2 – Continuous Integration

Whenever a developer creates a pull request or pushes code to the main branch, GitHub Actions starts the CI process.

The CI pipeline automatically:

1. Checks out source code.
2. Installs dependencies.
3. Performs static analysis.
4. Builds the application.
5. Runs automated tests.
6. Validates the AI model.
7. Performs security checks.

A failed critical test prevents the code from being deployed.

7. Stage 3 – Build

The system contains several components:

- Web/mobile frontend
- Backend APIs
- AI recommendation service
- Nutrition database service
- Wearable-device integration service

These components are built automatically.

Source Code



Install Dependencies



Build Application



Generate Artifacts



Build Successful?

8. Stage 4 – Automated Testing

Automated testing is essential because the platform processes personal health information and produces personalized recommendations.

Unit Testing

Tests individual functions such as:

- Nutrient calculation
- Calorie calculation
- User-profile processing
- Meal-plan generation
- Data validation
- Authentication

Tool: PyTest

API Testing

Tests APIs used by:

- Mobile applications
- Web applications
- Wearable devices
- AI services

Example:

Mobile App



REST API



Recommendation Engine



Database

Tool: Postman/Newman

Integration Testing

Tests interactions between system components.

Examples:

Wearable Device



Wearable API



Health Data Service



AI Recommendation Engine



Personalized Recommendation

UI Testing

Automated tests verify:

- Login
- User registration
- Health-profile entry
- Dietary preference selection
- Meal-plan display
- Nutrition tracking
- Progress dashboard

Tool: Selenium

9. AI/ML Model Validation

AI is a major component of the platform, so ML models must be validated before deployment.

The pipeline should test:

- Prediction accuracy
- Recommendation quality
- Model performance
- Data consistency
- Bias/fairness indicators
- Model version
- Input-data validation
- Output validation

Example:

Training Data



ML Model



Validation Dataset



Performance Evaluation



Quality Threshold?

/ \

YES NO



Deploy Reject

MLflow can be used for experiment tracking and model-version management.

A model that does not meet predefined quality thresholds should not be promoted to production.

10. Code Quality and Security

Security is particularly important because the platform handles sensitive health information.

SonarQube

Checks:

- Bugs
- Code smells
- Duplicated code
- Maintainability
- Security issues

Dependency Scanning

Checks third-party packages for known vulnerabilities.

Docker Security

Trivy scans container images before they are pushed to the production registry.

Secret Scanning

The pipeline checks for accidentally exposed:

- API keys
- Passwords
- Access tokens

- Cloud credentials

Secrets should be stored using **GitHub Secrets or a cloud secret manager**, not inside source code.

11. Docker Packaging

After successful testing, application components are packaged into Docker images.

Frontend



Docker Image

Backend



Docker Image

AI Service



Docker Image

Wearable Service



Docker Image

Docker provides:

- Environment consistency
- Portability
- Application isolation
- Easier deployment
- Easier rollback

12. Container Registry

The generated Docker images are stored in a container registry such as AWS ECR.

Images should use version tags:

nutrition-api:1.0.0

nutrition-api:1.0.1

nutrition-ai:2.1.0

This makes it possible to identify and restore previous versions.

13. Deployment Environments

The system should have three environments.

Development

Used for active development.

Developer



Development Environment

Staging

A production-like environment used for final validation.

CI Pipeline



Staging



Acceptance Testing



Security Testing

Production

The live environment used by actual users.

Staging



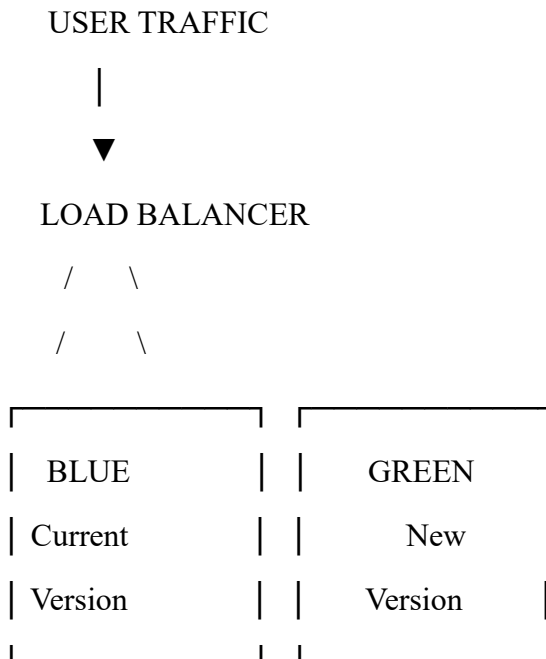
Approval



Production

14. Production Deployment Strategy

A **Blue-Green Deployment** strategy is recommended.



The new release is deployed to Green.

After testing:

Blue → Green

If the new version fails:

Green → Blue

This reduces downtime and provides fast recovery.

15. Security and Privacy Strategy

Because the application processes personal health information, security must be integrated throughout the pipeline.

Recommended practices

1. Encrypt data during transmission using HTTPS/TLS.
2. Encrypt sensitive data at rest.
3. Use role-based access control.
4. Store credentials in a secret manager.
5. Never commit passwords or API keys to GitHub.

6. Scan dependencies regularly.
7. Scan Docker images.
8. Perform security testing before production.
9. Maintain audit logs.
10. Apply the minimum-required access principle.
11. Separate development, staging, and production data.
12. Avoid using real patient/user health data in development and testing.

16. Monitoring

After deployment, the platform should be continuously monitored.

Prometheus

Collects:

- CPU utilization
- Memory utilization
- API response time
- Request rate
- Error rate
- Service availability

Grafana

Provides dashboards for:

System Health

|

|— API Performance

|— Application Errors

|— ML Service Status

|— Database Health

|— Infrastructure Metrics

ELK Stack

Centralizes application and infrastructure logs.

17. Notification Strategy

The pipeline sends notifications when important events occur.

Event	Notification
Build successful	Developer
Unit test failed	Developer
Security scan failed	DevOps/Security
ML validation failed	ML Team
Staging deployment successful	Team
Production deployment successful	Team
Production failure	DevOps/Operations
Rollback triggered	Team + Operations

Possible channels:

- Email
- Slack
- Microsoft Teams

18. Rollback Strategy

If the production version causes problems, the previous stable version is restored.

New Release



Production Monitoring



Problem Detected?

/ \

NO YES



Continue Rollback



Previous Stable

Version

Rollback triggers can include:

- High application error rate
- Failed health checks
- Severe performance degradation
- Service crashes
- Critical security vulnerability
- Failed production validation

19. Example CI/CD Workflow

Developer Commit



GitHub Pull Request



Code Review



Build



Unit Tests



API Tests



Integration Tests



UI Tests



SonarQube Analysis



Security Scanning



AI/ML Model Validation



Docker Build



Trivy Image Scan



Push to Container Registry



Deploy to Staging



Acceptance Testing



Manual Approval



Production Deployment



Monitoring



Success —————



Failure → Rollback ————

20. Pipeline Summary

Stage	Main Activity	Result
1	GitHub source control	Versioned code
2	CI	Automated integration
3	Build	Application artifacts
4	Unit/API/Integration tests	Verified functionality

5	Code quality/security	Secure code
6	AI/ML validation	Validated model
7	Docker packaging	Container image
8	Registry	Versioned image
9	Staging	Release candidate
10	Approval	Authorized release
11	Production	Live application
12	Monitoring	System visibility
13	Rollback	Recovery from failure

21. Justification

The proposed CI/CD pipeline is appropriate for the AI-Powered Personalized Nutrition and Health Recommendation Platform because the application combines AI/ML, mobile/web applications, wearable devices, APIs, cloud infrastructure, and sensitive health information.

GitHub Actions automates integration and delivery, while Docker and Kubernetes provide consistent and scalable deployment. Automated unit, API, integration, UI, security, and ML tests identify problems before production. SonarQube and vulnerability scanners improve software quality and security. ML validation prevents an inadequately performing model from being released. Staging provides a safe environment for final testing, while Blue-Green deployment and rollback improve availability and reliability.

Overall, the CI/CD pipeline enables faster, safer, repeatable, and controlled releases while supporting the platform's objectives of personalized nutrition, continuous monitoring, improved user engagement, and better health management.